

Assignment II: SDD, BDD, and TDD in AI-Assisted Software Development

Student Information

- **Name:** 阮清閒
- **Student ID:** 1123564
- **Course:** AI-assisted Software Development
- **Date:** 2026/05/24
- **Due Date:** 2026/05/31 23:59:59

1. Introduction

1.1 What is AI-Assisted Software Development?

AI-assisted software development refers to using AI tools like GitHub Copilot, ChatGPT, or Gemini can help developers during the software development process. These tools can generate code, suggest implementations, explain errors, write documentation, and even review code. The core idea is that instead of writing every single line of code manually, developers can collaborate with AI to speed up their workflow. That sounds great, but the problem is - if your description is vague, the output will be vague too. Therefore, "AI-assisted" doesn't mean "AI does everything". The AI still needs clear guidance from the developer, it doesn't read your mind, it only responds to what you actually tell it.

1.2 Why Clear Requirements Matter When Using AI

For example, if I ask an AI: *"Build a student grade calculator"*, the AI might:

- Forget to round the final score
- Not validate whether scores are within 0 - 100
- Use equal weights for all components instead of the correct weights

None of these are "wrong" from the AI's perspective, it just didn't know better. But they're all wrong from my perspective as the developer. That's why clear requirements are not optional when working with AI. They are the foundation of getting useful output.

1.3 Why SDD, BDD, and TDD Are Useful in the AI Era

SDD, BDD, and TDD are three complementary approaches that together help developers communicate precisely and evaluate rigorously:

- **SDD (Specification-Driven Development)** helps us define exactly what the system should do: inputs, outputs, rules, and constraints.
- **BDD (Behavior-Driven Development)** lets us describe how the system should *behave* from the user's perspective, using concrete scenarios.
- **TDD (Test-Driven Development)** pushes us to think about how to *verify* the system works, before we even start building.

These three approaches turn an unpredictable AI interaction into a structured development

workflow. You specify what you want (SDD), describe how it should behave (BDD), and verify that it works (TDD). This is a much more reliable way to work with AI than just hoping the output is correct.

2. Definition of SDD

SDD stands for **Specification-Driven Development**. The main idea is simple: before writing any code, you write down exactly what the software needs to do. It's like creating a detailed blueprint before building a house.

A well-written SDD specification typically contains the following elements:

Item	Description	Why It Matters
Goal	What problem is this software solving?	Helps everyone stay aligned on purpose
Functional Requirements	What specific tasks must the system perform?	Defines the scope of what needs to be built
Input	What data does the system receive?	Prevents ambiguity about valid inputs
Output	What does the system produce?	Ensures the result is well-defined
Constraints	What rules, limitations, or conditions apply?	Covers edge cases and boundaries
Acceptance Criteria	How do we verify the system is working correctly?	Provides a concrete checklist for validation

The reason SDD is useful is that when you feed a clear spec to an AI, it can generate code that's actually aligned with what you need. Without a spec, you might get working code that does the wrong thing.

3. SDD: Student Grade Calculator

3.1 Goal

To help students and instructors quickly understand overall performance, this system takes four academic scores (Assignment, Midterm Exam, Final Exam, and Project) as input to compute and appropriately round a final weighted score. It then outputs the final score and corresponding letter grade, while actively protecting against invalid inputs to ensure garbage data doesn't produce misleading results.

3.2 Functional Requirements

- The system must receive four numeric score values from the user.
- Each score is weighted and combined to compute a final score.
- The system must round the final score to one decimal place.
- Based on the rounded final score, the system determines and outputs a letter grade.
- If any input score falls outside the valid range (0-100), the system must notify the user that the input is invalid and should not proceed with the calculation.

3.3 Input

The system takes the following four inputs, each representing a numeric score between 0 and 100 (inclusive):

Input Field	Valid Range
Assignment score	0.0 - 100.0
Midterm Exam score	0.0 - 100.0
Final Exam score	0.0 - 100.0
Project score	0.0 - 100.0

Input constraint: All four scores must be present. The system should not attempt to calculate if any input is missing or invalid.

3.4 Output

After processing valid inputs, the system produces the following output:

Output	Format	Description
Final Score	Decimal, one decimal place	The weighted average of all four components
Letter Grade	Single character: A, B, C, D, or F	Grade assigned based on the final score

If any input is invalid, the output should be an error message instead, clearly indicating which input is problematic and what the valid range is.

3.5 Grade Rules

The final score is calculated using the following weighted formula:

$$\text{Final Score} = \text{Assignment} \times 0.30 + \text{Midterm} \times 0.20 + \text{Final Exam} \times 0.30 + \text{Project} \times 0.20$$

The weights reflect the relative importance of each component in the course:

- Assignment and Final Exam each carry 30% weight
- Midterm and Project each carry 20% weight
- Total weight: $0.30 + 0.20 + 0.30 + 0.20 = \mathbf{1.00}$ (100%)

After computing the raw final score, it is **rounded to one decimal place**. The rounded score is then mapped to a letter grade using the following table:

Final Score	Letter Grade
$90.0 \leq \text{score} \leq 100.0$	A
$80.0 \leq \text{score} < 90.0$	B
$70.0 \leq \text{score} < 80.0$	C
$60.0 \leq \text{score} < 70.0$	D
$\text{score} < 60.0$	F

Important boundary note: The boundaries are inclusive on the lower end.

3.6 Acceptance Criteria

The following criteria determine whether the system is working correctly:

1. **Correct weighted calculation:** Given valid inputs, the final score must be computed using the exact formula: $\text{Assignment} \times 0.30 + \text{Midterm} \times 0.20 + \text{Final Exam} \times 0.30 + \text{Project} \times 0.20$. Any deviation in weights should be considered a failure.
1. **Correct grade assignment at boundaries:** The system must assign the correct grade at boundary values. For example, a final score of exactly 80.0 must return grade B (not C), and a score of 89.9 must also return B (not A). The boundary behavior must be strictly consistent with the defined intervals.
1. **Proper handling of out-of-range inputs:** If any single score is below 0 or above 100, the system must not proceed with the calculation. Instead, it should return or display an appropriate error message.
1. **Rounding is applied before grading:** The final score must be rounded to one decimal place first, and then the letter grade is determined from the rounded value - not from the raw calculated value.

4. Definition of BDD

BDD stands for **Behavior-Driven Development**. While SDD focuses on what the system should do from a technical angle, BDD focuses on *how the system should behave* from the perspective of the user.

The most recognizable feature of BDD is the **Given-When-Then** format:

Keyword	Role	What it describes
Given	Pre-condition / Setup	The initial state or context before any action
When	Trigger / Action	The event or action that the user or system performs
Then	Expected Result	What the system should produce or do after the action

This format is powerful because it reads like plain English. Even non-technical team members like designers, product managers, or QA testers can read and understand it. In an AI context, you can paste BDD scenarios directly into a prompt, and the AI can understand what behavior to implement, without you needing to explain the logic from scratch.

BDD is especially useful when there are multiple edge cases or user stories, because each scenario is self-contained and unambiguous.

5. BDD: Student Grade Calculator

Scenario 1: Student receives a grade B

```
Given the assignment score is 85
And   the midterm exam score is 78
And   the final exam score is 82
And   the project score is 80
When  the system calculates the final score
Then  the final score should be 81.7
And   the letter grade should be B
```

Verification:

```
Final Score = 85 × 0.30 + 78 × 0.20 + 82 × 0.30 + 80 × 0.20
            = 25.5 + 15.6 + 24.6 + 16.0
            = 81.7
```

Grade: **B** ($80.0 \leq 81.7 < 90.0$) ✓

Scenario 2: Student receives a grade F

```
Given the assignment score is 62
And   the midterm exam score is 40
And   the final exam score is 38
And   the project score is 65
When  the system calculates the final score
Then  the final score should be 51.0
And   the letter grade should be F
```

Verification:

```
Final Score = 62 × 0.30 + 40 × 0.20 + 38 × 0.30 + 65 × 0.20
            = 18.6 + 8.0 + 11.4 + 13.0
            = 51.0
```

Grade: **F** ($51.0 < 60.0$) ✓

Scenario 3: System rejects an invalid input score

```
Given the assignment score is 105
And   the midterm exam score is 70
And   the final exam score is 68
And   the project score is 75
When  the system attempts to calculate the final score
Then  the system should not produce a final score
And   an error message should be displayed indicating that the assignment
score is out of valid range
```

Scenario 4: Student lands exactly on a grade boundary

```
Given the assignment score is 68
And   the midterm exam score is 72
And   the final exam score is 70
And   the project score is 71
When  the system calculates the final weighted score
Then  the final score should be 70.0
And   the letter grade should be C
And   the system should not assign grade D
```

Verification:

```
Final Score = 68 × 0.30 + 72 × 0.20 + 70 × 0.30 + 71 × 0.20
            = 20.4 + 14.4 + 21.0 + 14.2
            = 70.0
```

Grade: C ($70.0 \leq \text{score} < 80.0$) ✓

6. Definition of TDD

TDD stands for **Test-Driven Development**. The key idea is a bit counterintuitive at first: instead of writing the code first and then testing it, you write the *tests first*, and then write just enough code to make those tests pass.

The TDD process follows a simple cycle:

Phase	Color	What Happens
Write a failing test	Red	Write a test that describes desired behavior. It fails immediately because the feature doesn't exist yet. That's expected.
Make the test pass	Green	Write the simplest, most minimal code that makes the test pass. Don't over-engineer, just make it pass.
Improve the code	Refactor	Clean up the code, remove duplication, improve readability - without breaking any existing tests.

This cycle repeats for every new test case. The result is code that is always tested, always working for every scenario you've defined.

In the context of AI-assisted development, TDD is a great way to *verify* AI-generated code. You can write test cases based on your specs and BDD scenarios, then run them against whatever the AI produces. If the tests fail, you know the AI output isn't correct. This makes TDD a powerful quality-control tool in an AI workflow.

In this assignment, I won't be writing actual code, just the test case designs.

7. TDD: Student Grade Calculator

Scenario 1: Normal Test Cases

These test cases cover typical, everyday input scenarios where all scores are valid and the results are straightforward.

Test Case 1: Student achieves grade C

Input

- Assignment: 72
- Midterm: 65

- Final Exam: 74
- Project: 70

Expected Calculation

$$\begin{aligned}\text{Final Score} &= 72 \times 0.30 + 65 \times 0.20 + 74 \times 0.30 + 70 \times 0.20 \\ &= 21.6 + 13.0 + 22.2 + 14.0 \\ &= 70.8\end{aligned}$$

Expected Output

- Final Score: **70.8**
- Letter Grade: **C**

Test Case 2: Student achieves grade D

Input

- Assignment: 60
- Midterm: 58
- Final Exam: 65
- Project: 62

Expected Calculation

$$\begin{aligned}\text{Final Score} &= 60 \times 0.30 + 58 \times 0.20 + 65 \times 0.30 + 62 \times 0.20 \\ &= 18.0 + 11.6 + 19.5 + 12.4 \\ &= 61.5\end{aligned}$$

Expected Output

- Final Score: **61.5**
- Letter Grade: **D**

Scenario 2: Boundary Test Cases

These test cases specifically target score values at or near the boundaries of each grade range. Boundary cases are important because they're the most common place for off-by-one errors.

Test Case 1: Exactly at the boundary between B and A (score = 90.0)

Input

- Assignment: 90
- Midterm: 90
- Final Exam: 90
- Project: 90

Expected Calculation

$$\begin{aligned}\text{Final Score} &= 90 \times 0.30 + 90 \times 0.20 + 90 \times 0.30 + 90 \times 0.20 \\ &= 27.0 + 18.0 + 27.0 + 18.0 \\ &= 90.0\end{aligned}$$

Expected Output

- Final Score: **90.0**
- Letter Grade: **A** (since $90.0 \leq \text{score} \leq 100.0$)

Test Case 2: Just below the D/F boundary (score = 59.9)

Input

- Assignment: 58
- Midterm: 62
- Final Exam: 58
- Project: 62

Expected Calculation

$$\begin{aligned}\text{Final Score} &= 58 \times 0.30 + 62 \times 0.20 + 58 \times 0.30 + 62 \times 0.20 \\ &= 17.4 + 12.4 + 17.4 + 12.4 \\ &= 59.6\end{aligned}$$

Expected Output

- Final Score: **59.6**
- Letter Grade: **F** (since $59.6 < 60.0$)

Scenario 3: Invalid Input Test Cases

These test cases check how the system handles inputs that are outside the valid range (0-100). The system should detect these cases and return an error instead of computing a grade.

Test Case 1: Negative score in one component

Input

- Assignment: -10
- Midterm: 75
- Final Exam: 80
- Project: 70

Expected Behavior

The system should detect that the Assignment score is below 0.

Expected Output

- Final Score: **Not calculated**
- Letter Grade: **Not assigned**
- Error Message: *"Invalid input: Assignment score must be between 0 and 100."*

Test Case 2: Score greater than 100 in Final Exam

Input

- Assignment: 88
- Midterm: 76
- Final Exam: 110
- Project: 82

Expected Behavior

The system should detect that the Final Exam score exceeds 100.

Expected Output

- Final Score: **Not calculated**
- Letter Grade: **Not assigned**
- Error Message: *"Invalid input: Final Exam score must be between 0 and 100."*

8. Comparison of SDD, BDD, and TDD

Item	SDD	BDD	TDD
Full Name	Specification-Driven Development	Behavior-Driven Development	Test-Driven Development
Main Focus	Define the system clearly before building	Describe behavior through concrete examples	Verify correctness by testing before coding
Core Question	<i>What should the system do?</i>	<i>How should it behave in real scenarios?</i>	<i>Does it actually work correctly?</i>
Typical Format	Structured spec documents (inputs, outputs, rules)	Given-When-Then scenario format	Test case tables with inputs and expected outputs
Who Uses It	Developers, architects, PMs	All stakeholders including non-technical members	Developers and QA testers
When to Apply	Before development begins	Before or during development	Before and throughout implementation
Relationship to AI	Provides clear prompt specifications	Provides behavioral examples for AI to follow	Provides test cases to verify AI-generated code
Strength	Leaves little ambiguity about what to build	Bridges gap between technical and non-technical teams	Catches implementation errors systematically
Weakness	Can become outdated if not maintained	Large projects can have too many scenarios to manage	Writing tests first is unfamiliar for many developers

In practice, these three approaches complement each other well. SDD sets the foundation, BDD translates it into user-facing behavior, and TDD validates that everything works as expected.

9. Reflection

When I first read about SDD, BDD, and TDD, they seemed like three separate techniques with little connection. But working through this assignment changed that perspective, I now see them as a natural progression from "what we want" to "how it should behave" to "does it actually work."

Of the three, I found **BDD the easiest to understand**. The Given-When-Then format feels natural. You're basically telling a story: here's the situation, here's what happened, here's what should result. It reminds me of how I'd describe a bug to a friend, not in code, but in plain language.

SDD was harder, mostly because writing good acceptance criteria requires you to think very precisely. It's not enough to say "grades should be correct" - you have to define what "correct" means in every edge case. That kind of thinking takes practice.

TDD felt the most abstract at first, but it's actually the most powerful when working with AI. Here's why: AI-generated code often *looks* correct but has subtle mistakes, especially at boundaries (like "should 89.9 be B or A?"). If you have test cases ready, you can just run them and immediately see if the AI's output actually behaves the way you intended.

When it comes to **working with AI coding tools**, I think TDD is the most directly useful. After generating code with AI, you don't have to manually trace through every line, you run the tests and they tell you what's wrong. It's an efficient quality check.

That said, **SDD is probably the most important for avoiding problems in the first place**. If I ask an AI "build a grade calculator," I might get something that works but doesn't round correctly, or uses wrong grade boundaries. But if I include a detailed spec - "round to 1 decimal, grade A is 90.0 to 100.0 inclusive, etc." - the AI has much less room to make those kinds of mistakes.

SDD directly solves the "vague prompt" problem. When people use AI without a spec, they tend to write prompts like "calculate the grade", and the AI fills in all the missing details on its own, which may or may not match what you actually want. SDD forces you to answer all those details yourself before even opening the AI tool: What are the valid input ranges? What formula is used? How many decimal places? What are the exact grade boundaries? Once you've written all of that in a spec, your prompt naturally becomes precise. You're not hoping the AI guesses right - you're telling it exactly what "right" looks like.

BDD helps describe user expectations in a way that is concrete and grounded in real examples. Instead of writing abstract rules like "the system should handle invalid inputs," BDD pushes you to write an actual scenario: given a score of -10, when the user submits, then the system shows an error. That's not just a rule, it's a story told from the user's point of view. It captures not just *what* the system does, but *when* and *why* it does it. This is especially useful when different people have different interpretations of the same requirement. A BDD scenario leaves almost no room for misinterpretation, because it's tied to a specific, concrete example that everyone can read and verify. If I were working on a real software project with AI tools, my workflow would look something like this: start with SDD to define requirements clearly, use BDD to specify expected behaviors in key scenarios, use those BDD scenarios as direct prompts or context when asking AI to implement the feature, and finally use TDD test cases to verify the output. This three-layer approach means AI isn't just a code generator - it's a collaborator that I can guide, evaluate, and correct systematically.

10. References / AI Tool Usage

Claude (Anthropic) was used to assist in understanding the definitions of SDD, TDD and phrasing of this report. All score combinations, calculations, acceptance criteria, BDD scenarios, and test case designs are my own work. All mathematical calculations were manually verified. The reflection section was written entirely in my own words, based on my experience completing this assignment.

Reference Materials

- North, D. (2006). *Introducing BDD* (blog post). <https://dannorth.net/introducing-bdd/>
- Cucumber Documentation - Gherkin Reference: <https://cucumber.io/docs/gherkin/reference/>